Smart Manufacturing Multi-Agent System (MAS)

Theoretical Foundations & Architectural Pivot: Autonomous Closed-Loop Prescriptive Maintenance

Course: AIM801 Project Elective (Semester 7) | **Domain:** Cyber-Physical Systems, RecSys & Explainable AI | **Date:** August 2026

Executive Overview: This technical specification establishes the mathematical, theoretical, and architectural foundation for pivoting the Smart Manufacturing MAS from an **open-loop Human-in-the-Loop (HITL) advisory platform** to an **autonomous, closed-loop cyber-physical prescriptive maintenance controller**. We provide complete theoretical explanations for **SHAP (Cooperative Game Theory)**, **Case-Based Reasoning (Cognitive Case Retrieval)**, **LambdaMART / LGBMRanker (Learning-to-Rank)**, and the **Finite State Machine Execution & Feedback Loop**.

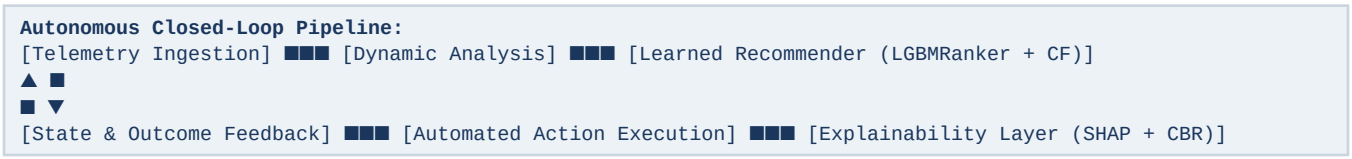
1. Recap of Previous Semester (Sem 6) & Core Bottlenecks

In Semester 6, we implemented a 5-agent architecture (DataLoaderAgent, PreprocessingAgent, DynamicAnalysisAgent, OptimizationAgent, and LLMPlannerAgent / RulesFirstPlannerAgent) across a 3-tier intelligence hierarchy. While data ingestion and model benchmarking were successful, operational evaluation identified three fundamental constraints:

- **Heavy Human-in-the-Loop (HITL) Overhead:** Blocking operator confirmation prompts occurred at problem-type selection, contamination parameter setting, model retries, and action sign-offs, creating human latency bottlenecks.
- **Static Heuristic Action Generation:** The OptimizationAgent mapped predictions to static strings (_action_plan_from_score) via rigid thresholding, unable to capture multi-sensor compound interactions.
- **Open-Loop / One-Way Pipeline:** The platform halted at advisory text reports without an execution mechanism or a feedback store to monitor recovery and adapt over time.

2. Proposed Pivot (Sem 7): Autonomous Closed-Loop Architecture

The Semester 7 architecture replaces static heuristics and human gates with a continuous cyber-physical learning and control loop:



3. Deep Theoretical Foundations of Newly Introduced Concepts

To replace human approval gates safely and eliminate black-box decision risks, we introduce rigorous mathematical and cognitive AI foundations across the pipeline.

A. SHAP (SHapley Additive exPlanations): Theory, Axioms, & TreeSHAP

1. Mathematical Formulation: Derived from Cooperative Game Theory (Shapley, 1953) and adapted to machine learning by Lundberg & Lee (2017), SHAP computes the marginal contribution of each feature *i* across all possible feature subsets $S \subseteq F \setminus \{i\}$:

```
Classical Shapley Value Formulation:
ϕ_i(x) = Σ_{S ⊆ F \ {i}} [ |S|! (|F| - |S| - 1)! / |F|! ] · [ f_x(S ∪ {i}) - f_x(S) ]
where |F| is total features, S is a coalition of features, and f_x(S) is the model output conditioned on S.
```

2. The Four Fundamental Axiomatic Guarantees:

- **Local Accuracy (Efficiency):** The sum of feature attributions equals the difference between model output and expected baseline: $\Sigma_{i=1}^M \phi_i(x) = f(x) - E[f(X)]$.
- **Missingness:** A feature with no influence in any coalition receives exactly zero attribution: x_i *missing* ■ $\phi_i(x) = 0$.
- **Consistency (Monotonicity):** If a model changes such that feature *i*'s marginal contribution increases or stays equal for all coalitions, its Shapley value cannot decrease.
- **Additivity:** For ensemble models $f = \Sigma\, w_k f_k$, the ensemble Shapley value is the weighted sum of individual tree Shapley values: $\phi_i(f) = \Sigma\, w_k \phi_i(f_k)$.

3. Why TreeSHAP is Essential for Real-Time Prescriptive Manufacturing: Exact brute-force Shapley calculation requires exponential time $O(M \cdot 2^{|F|})$, which is impossible during real-time plant operations. **TreeSHAP** evaluates all feature permutations simultaneously in polynomial time $O(T \cdot L \cdot D^2)$ (where T is number of trees, L is maximum leaves, and D is maximum tree depth). This allows our system to compute exact local attribution vectors for ranker decisions in sub-second latency.

B. Case-Based Reasoning (CBR): Theory, Metric Spaces, & Peer Retrieval

1. Conceptual Foundation: Grounded in Cognitive Science (Aamodt & Plaza, 1994), CBR operates on the premise that *similar physical problems possess similar optimal solutions*. While SHAP explains 'which features drove the score', CBR answers 'what happened when this exact action was executed in identical historical operating conditions'.

```
Archetype Distance Metric & Precedent Utility Weighting:
1. Multi-Sensor Archetype Cosine Distance: d(m_target, m_k) = 1 - ( v_target · v_k ) / ( ||v_target|| · ||v_k|| )
2. Kernel-Weighted Peer Utility: U_est(m_target, a) = [ Σ_{k ∈ N} exp(-λ d(m_target, m_k)) · U(m_k, a) ] / [ Σ_{k ∈ N} exp(-λ d(m_target, m_k)) ]
where v is the normalized multi-sensor feature vector, and U(m_k, a) is the observed historical post-intervention recovery.
```

2. The 4R Cycle in Industrial Prescriptive Maintenance:

- **Retrieve:** Query the historical asset database for the top-*K* nearest neighbor machine states using multi-sensor cosine distance $d(m_{target}, m_k)$.
- **Reuse:** Extract the intervention strategy that historically maximized recovery rate and minimized downtime in those peer machines.
- **Revise:** Tailor the action plan to the specific asset's operational constraints (e.g. current production schedule, cost budget).
- **Retain:** Log the post-intervention recovery trajectory back into the asset database to enrich future peer retrievals.

C. Learning-to-Rank (LambdaMART / LGBMRanker): Theory & Gradient Weighting

1. Why Classification Fails for Action Selection: Standard multi-class classification treats actions as mutually exclusive, unordered classes. In reality, maintenance decisions are **listwise rankings** where multiple actions may be viable, but one provides superior net economic recovery.

LambdaMART Pairwise Gradient Formulation:

$$\lambda_{ij} = \partial C_{ij} / \partial s_i = \left[-\sigma / \left(1 + \exp(\sigma(s_i - s_j)) \right) \right] \cdot | \Delta \text{NDCG}_{ij} |$$

where s_i, s_j are ranker scores for actions a_i, a_j, and $|\Delta \text{NDCG}_{ij}|$ is the exact NDCG change if positions of a_i and a_j are swapped.

2. Direct Optimization of Listwise Utility (NDCG@k): By scaling the gradient directly by $|\Delta NDCG_{ij}|$, LambdaMART focuses gradient updates on mistakes occurring at the top of the recommendation list (high ranks), ensuring the top-1 action is optimal.

4. Detailed Breakdown of the Two Autonomous Modules

■ Module 1: Self-Explaining Interpretability Layer (SHAP + Case-Based Reasoning)

The Explainability Layer bridges the gap between complex non-linear models and regulatory industrial auditability by synthesizing mathematical proof with empirical evidence:

Dual-Perspective Explainability Synthesis:
1. **Local Quantitative Attribution (TreeSHAP):** Computes exact local Shapley values for the candidate action. Discovers compound risks (e.g. vibration = 2.4 mm/s is safe in isolation, but coupled with 4 prior failures and ambient temperature > 35°C, it drives 84% of the high-priority score).
2. **Empirical Historical Precedent (CBR):** Retrieves the 3 nearest peer machines from the archetype embedding space. Validates real-world outcomes: *'In 3 peer machines (M12, M18, M27) with similar vibration profiles, executing Action A restored nominal performance in 3.5 hrs with $0 downtime, whereas Action B caused bearing seizure within 48 hrs.'*
3. **Automated Audit Generation:** Formats SHAP vectors and CBR precedents into structured JSON and passes them to the LLM Reflexion loop to generate timestamped compliance summaries.

■ Module 2: Autonomous Execution State Machine Engine

The Autonomous Execution Engine converts the MAS from an advisory reporting tool into an **active cyber-physical closed-loop controller**:

Finite State Machine (FSM) Lifecycle & Mathematical Feedback:
• **IDLE:** Continuous baseline telemetry monitoring.
• **ACTION_TRIGGERED:** Anomaly detected; LGBMRanker outputs ranked action candidates.
• **POLICY_VALIDATION:** Evaluates confidence score against threshold τ_exec and validates safety constraints (maximum downtime, cost budget, interlock status).
• **DISPATCHED:** Autonomous dispatch of programmatic control adjustments (PLC/actuator commands) or automated work tickets.
• **EXECUTING:** Action active on physical or simulated asset.
• **VERIFYING_RECOVERY:** Tracks physical recovery over observation horizon *t ... t+H*:
  Δ **Recovery = || x_t - x_nominal || - || x_{t+H} - x_nominal ||**
• **COMPLETED / ESCALATED:** Anomaly successfully arrested OR automated fallback triggered.
• **FEEDBACK_LOGGED:** Saves transition tuple *(s_t, a_t, r_t, s_{t+H})* to Asset History Store, where reward is:
  r_t = (Avoided Downtime Cost) - (Action Cost) - Penalty(Recovery Delay)

5. Systematic Comparison: Semester 6 vs. Semester 7 Pivot

Dimension	Last Semester (Sem 6)	Proposed This Semester (Sem 7 Pivot)
System Loop	**Open-loop:** Stops after generating advisory CSV/JSON reports.	**Closed-loop:** Prediction ■■■ Ranking ■■■ Dispatch ■■■ Recovery Verification ■■■ Feedback Ingestion.
Operator Role	**Heavy HITL:** Blocking approval gates at every stage (problem type, contamination, retry, action sign-off).	**Autonomous Execution:** Direct dispatch with confidence thresholds (τ_exec) and automated safety fallbacks.
Action Generation	**Static Heuristics:** Hardcoded if/else thresholds (_action_plan_from_score).	**Learned Recommender:** LGBMRanker (LambdaMART) optimizing NDCG@k per asset query group.
Cross-Asset Signal	**None:** Assets scored strictly in isolation.	**Collaborative Filtering:** Cosine similarity over multi-sensor archetype embeddings transfers peer histories.
Action Justification	**Generic Templates:** Static strings with top-3 global feature importances.	**Dual-Pillar Explainability:** Local TreeSHAP attribution + Case-Based Reasoning peer incident precedents.
Execution Engine	**None:** Relies on external human technicians reading text reports.	**Finite State Machine:** Automated lifecycle management, recovery verification, and closed-loop feedback.
Evaluation Metrics	**Standard ML:** Balanced Accuracy, R^2, MSE, Anomaly Contamination Rate.	**RecSys & Economic:** NDCG@k, Precision@k, Net Cost Saved ($), Recovery Delta, MTBF.

6. Evaluation Metrics Framework & Industrial Economics

To rigorously validate the closed-loop system, evaluation expands beyond predictive ML metrics to ranking utility and industrial financial return:

- **Normalized Discounted Cumulative Gain (NDCG@k):** Measures listwise recommendation quality: $NDCG@k = DCG@k / IDCG@k$, where $DCG@k = \Sigma_{i=1}^{k} (2^{rel_i} - 1) / log_2(i + 1)$.
- **Precision@k:** Proportion of top-k recommended actions that successfully arrested machine degradation within horizon H.
- **Industrial Net Cost Saved:** $Net\ Savings = (Avoided\ Unplanned\ Downtime\ Cost) - (Intervention\ Execution\ Cost)$.
- **Mean Time Between Failures (MTBF):** Quantifies extended machine lifespan achieved through proactive, autonomous intervention.

7. Semester 7 Implementation Roadmap & Codebase Mapping

Phase 1: Recommender & Collaborative Filtering Engine (Weeks 1-3)
• Refactor agents/optimization_agent.py to formulate query groups $q = (Machine\_ID, t)$.
• Train LGBMRanker optimizing NDCG@k and build multi-sensor archetype cosine similarity matrix.

Phase 2: Self-Explaining Interpretability Layer (Weeks 4-6)
• Create utils/explainability.py integrating shap.TreeExplainer on ranker and diagnostic models.
• Build Case-Based Reasoning peer precedent retriever and format outputs for Cloud LLM Reflexion summaries.

Phase 3: Autonomous Execution State Machine & Feedback Store (Weeks 7-9)
• Implement agents/execution_agent.py Finite State Machine with confidence threshold gates.
• Build cyber-physical digital-twin simulation environment for post-action telemetry recovery ($t \dots t+H$).
• Implement utils/feedback_store.py for online experience replay and model adaptation.

Phase 4: Web Dashboard Integration & Benchmark Validation (Weeks 10-12)
• Extend FastAPI web dashboard (webapp/app.py) with live state machine visualization and ranking curves.
• Run comprehensive benchmark evaluations across NDCG@k, Precision@k, and Net Cost Saved ($).